Отчёт по лабораторной работе №23

по курсу «Фундаментальная информатика и информационные технологии».

Выполнил студент группы **М8О-114БВ-25 Сабирзянов Адель Рамилевич** № по списку 20.

Контакты: sabiadel386@gmail.com

Работа выполнена: «5» апреля 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «9» апреля 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. Тема

Динамические структуры данных. Деревья общего вида.

2. Цель работы

Изучение способов представления деревьев общего вида в памяти компьютера, освоение рекурсивных алгоритмов обхода деревьев и навыков управления динамической памятью в языке Си.

3. Задание

Реализовать программу для работы с деревом общего вида, поддерживающую следующие операции:

1. Добавление узла в дерево (в корень или к указанному родителю).
2. Визуализация структуры дерева с учетом вложенности (глубины).
3. **Специфическая задача (Вар. 20):** Найти максимальное значение среди всех узлов дерева и определить глубину, на которой оно находится.
4. Удаление узла вместе со всем его поддеревом.
5. Корректная очистка всей памяти при выходе из программы.

Данные в узлах должны быть представлены типом enum.

4. Оборудование

Процессор: 13th Gen Intel(R) Core(TM) i5-13420H @ 2.10 GHz, ОП 8 ГБ, SSD 1 ТБ

5. Программное обеспечение

Операционная система: Ubuntu 24.04 LTS (WSL), компилятор gcc версия 13.2.0, редактор VS Code.

6. Идея, метод, алгоритм решения задачи

Для реализации дерева используется структура LCRS (Left-Child Right-Sibling). Каждый узел содержит значение типа enum и два указателя: son (на первого ребенка) и brother (на следующего соседа того же уровня).

- **Поиск максимума:** Рекурсивный обход в глубину (DFS). При переходе к son счетчик глубины увеличивается, при переходе к brother — остается прежним.
- **Удаление:** Рекурсивное освобождение памяти через free(), начиная с самых глубоких узлов (постфиксный обход).

7. Сценарий выполнения работы

1. Описание структуры Node и перечисления NodeType.
2. Реализация функции поиска родителя findNode для корректного построения дерева.
3. Реализация функции findMaxDepth с передачей параметров по указателю для сохранения результата.
4. Создание текстового интерфейса (меню) для взаимодействия с пользователем.
5. Написание функции clearSubtree для предотвращения утечек памяти.

Текст программы:

```
#include <stdio.h>

#include <stdlib.h>



typedef enum {

    VAL_0 = 0,

    VAL_1,

    VAL_2,

    VAL_3,

    VAL_4,

    VAL_5,

    VAL_10 = 10,

    VAL_50 = 50,

    VAL_100 = 100

} NodeType;



typedef struct Node {
```

```
    NodeType data;
    struct Node *son;
    struct Node *brother;
} Node;

// Создание нового узла
Node* createNode(NodeType value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) return NULL;
    newNode->data = value;
    newNode->son = NULL;
    newNode->brother = NULL;
    return newNode;
}

// Поиск узла по значению перечисления
Node* findNode(Node* root, NodeType value) {
    if (root == NULL) return NULL;
    if (root->data == value) return root;

    Node* res = findNode(root->son, value);
    if (res) return res;

    return findNode(root->brother, value);
}
```

```
// Добавление дочернего узла
void addSon(Node* parent, NodeType value) {
    if (!parent) return;
    Node* newNode = createNode(value);
    if (parent->son == NULL) {
        parent->son = newNode;
    } else {
        Node* temp = parent->son;
        while (temp->brother != NULL) {
            temp = temp->brother;
        }
        temp->brother = newNode;
    }
}

// Визуализация структуры дерева
void printTree(Node* root, int depth) {
    if (root == NULL) return;

    for (int i = 0; i < depth; i++) printf("  ");
    printf("%d\n", (int)root->data); // Выводим числовое значение enum
    printTree(root->son, depth + 1);
    printTree(root->brother, depth);
}

// Поиск максимума и его глубины
```

```
void findMaxDepth(Node* root, int currentDepth, NodeType* maxVal, int* depthOfMax) {
    if (root == NULL) return;

    // Сравниваем значения enum как целые числа
    if (root->data > *maxVal) {
        *maxVal = root->data;
        *depthOfMax = currentDepth;
    }

    findMaxDepth(root->son, currentDepth + 1, maxVal, depthOfMax);
    findMaxDepth(root->brother, currentDepth, maxVal, depthOfMax);
}

// Рекурсивное удаление поддерева
void clearSubtree(Node* target) {
    if (target == NULL) return;
    clearSubtree(target->son);
    free(target);
}

// Удаление узла и его ветки
Node* deleteNode(Node* root, NodeType value) {
    if (root == NULL) return NULL;

    if (root->data == value) {
        Node* nextRoot = root->brother;
```

```
    clearSubtree(root->son);
    free(root);
    return nextRoot;
}

if (root->son && root->son->data == value) {
    Node* toDelete = root->son;
    root->son = toDelete->brother;
    clearSubtree(toDelete->son);
    free(toDelete);
    return root;
}

Node* curr = root->son;
while (curr && curr->brother) {
    if (curr->brother->data == value) {
        Node* toDelete = curr->brother;
        curr->brother = toDelete->brother;
        clearSubtree(toDelete->son);
        free(toDelete);
        return root;
    }
    curr = curr->brother;
}

deleteNode(root->son, value);
```

```
    deleteNode(root->brother, value);
    return root;
}

int main() {
    Node* root = NULL;
    int choice;
    int inputVal, parentVal;

    while (1) {
        printf("\n--- GENERAL TREE (ENUM DATA) - VAR 20 ---\n");
        printf("1. Add Node\n");
        printf("2. Print Tree\n");
        printf("3. Find Max & Depth (Task 20)\n");
        printf("4. Delete Node & Subtree\n");
        printf("5. Exit\n");
        printf("Select action: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input! Enter a number.\n");
            while(getchar() != '\n');
            continue;
        }

        if (choice == 1) {
            if (root == NULL) {
```

```
        printf("Enter root value (as integer): ");
        scanf("%d", &inputVal);
        root = createNode((NodeType)inputVal);
        printf("Root created.\n");
    } else {
        printf("Enter PARENT and NEW node values: ");
        if (scanf("%d %d", &parentVal, &inputVal) == 2) {
            Node* parent = findNode(root, (NodeType)parentVal);
            if (parent) {
                addSon(parent, (NodeType)inputVal);
                printf("Node added.\n");
            } else {
                printf("Parent not found!\n");
            }
        }
    }
} else if (choice == 2) {
    if (!root) printf("Tree is empty.\n");
    else printTree(root, 0);
} else if (choice == 3) {
    if (!root) printf("Tree is empty.\n");
    else {
        NodeType mVal = root->data;
        int dMax = 0;
        findMaxDepth(root, 0, &mVal, &dMax);
        printf("Max value: %d at Depth: %d\n", (int)mVal, dMax);
```

```
            }
        } else if (choice == 4) {
            if (!root) printf("Tree is empty.\n");
            else {
                printf("Enter value to delete: ");
                scanf("%d", &inputVal);
                root = deleteNode(root, (NodeType)inputVal);
                printf("Deleted (if existed).\n");
            }
        } else if (choice == 5) {
            if (root) {
                clearSubtree(root->son);
                free(root);
            }
            printf("Exit. Memory cleared.\n");
            break;
        }
    }
    return 0;
}
```

Соображения по тестированию:

Проверка пустого дерева: Выполнялся запуск функций поиска максимума и печати сразу после старта. Программа корректно вывела сообщение Tree is empty и не завершилась аварийно.

Тест корректности: Было построено дерево глубиной 3. Максимальное значение было размещено на среднем уровне. Скрипт верно определил и само значение, и его глубину (отсчет от 0 для корня).

Тест рекурсивного удаления: Был создан узел с несколькими потомками. При удалении этого узла программа успешно освободила память и из дочерних узлов, что подтверждено отсутствием их данных при повторном выводе дерева.

Устойчивость ввода: В поле выбора действий вводились символьные значения (буквы). Благодаря очистке буфера stdin, программа не вошла в бесконечный цикл, вывела ошибку и предложила повторить ввод.

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола

adel@MAI-LAPTOP:~/lab22$ gcc solution.c -o solution

adel@MAI-LAPTOP:~/lab22$ ./solution

--- GENERAL TREE (ENUM DATA) - VAR 20 ---

1. Add Node

2. Print Tree

3. Find Max & Depth (Task 20)

...

Select action: 1

Enter root value (as integer): 10

Root created.

Select action: 1

Enter PARENT and NEW node values: 10 100

Node added.

Select action: 3

Max value: 100 at Depth: 1**9. Дневник отладки**

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. Замечания автора по существу работы

Реализация дерева через LCRS является наиболее экономной по памяти для деревьев с произвольным количеством потомков.

11. Выводы

В ходе работы были освоены принципы построения и обхода деревьев общего вида. Реализован алгоритм поиска экстремума на дереве с вычислением его глубины. Закреплены навыки работы с динамической памятью и рекурсией в языке Си.

Подпись студента _____